

From JSON to Postgres

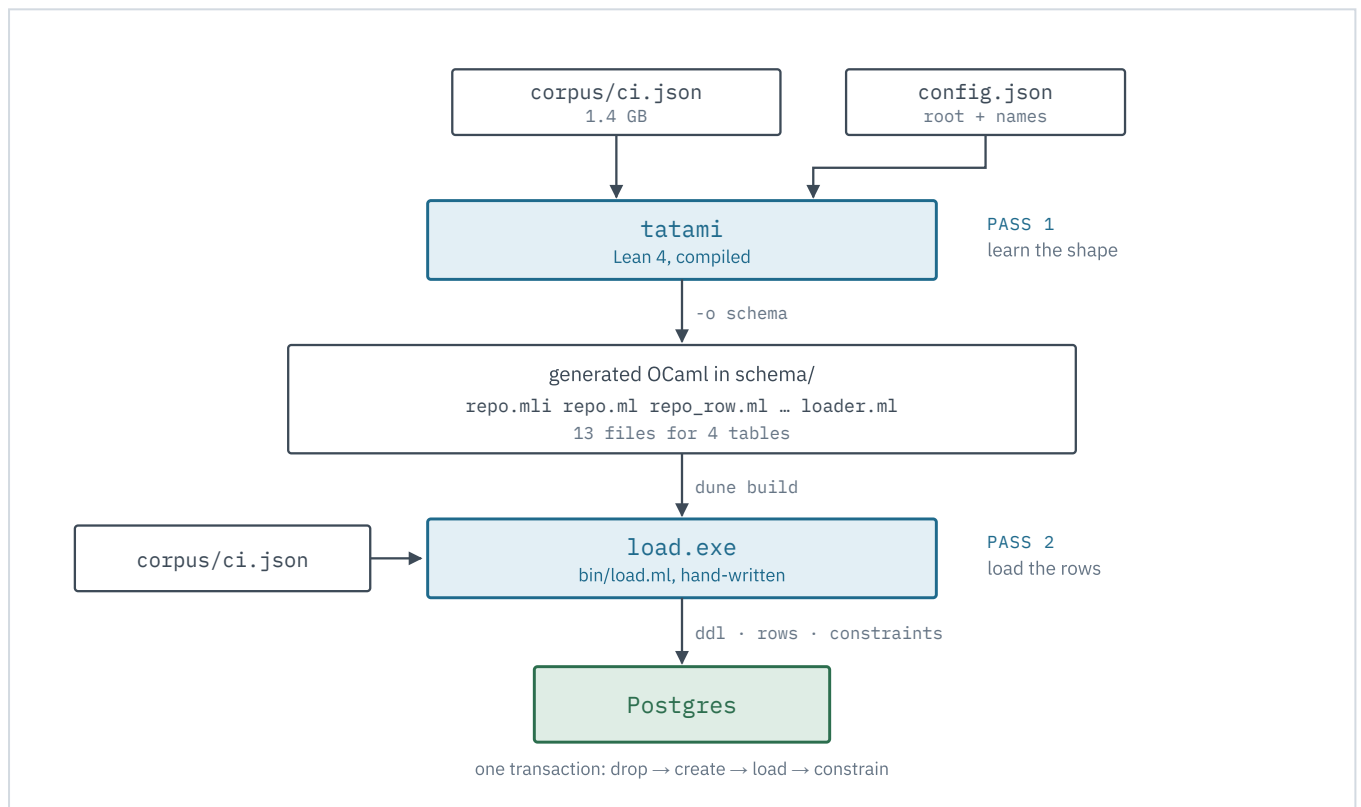
A 1.4 GB JSON corpus becomes four populated Postgres tables by way of generated OCaml. This replaces the earlier pipeline note, which described a design — Lean writing `.sql`, a shell script running `COPY` — that is no longer what the code does.

Generator Lean 4.33.0, no Mathlib **Loader** OCaml 5.5.0, pgx **Corpus** corpus/ci.json, 1.4 GB
docs/phase_1/pipeline.html ·rebuild with make -C docs pipeline

01

Two programs, two passes

The corpus is read twice, by two different programs, for two different purposes. The first pass produces *types*; the second produces *rows*. Nothing is carried between them except the generated code.

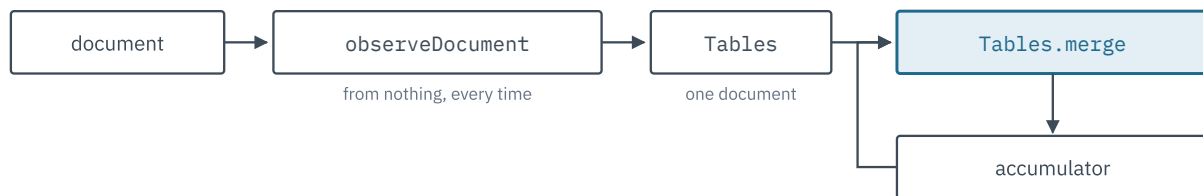


The Lean binary writes OCaml, not SQL. It *decides* the SQL — `ddl0f` and `constraints0f` compose the `create table` and `alter table` text — but emits it as OCaml string constants inside `loader.ml`. Nothing writes a `.sql` file, and there is no schema file to apply before loading.

02

Pass 1 — inference

Each document is observed *from nothing* and the results merged, rather than threaded through a shared accumulator. That is what makes the schema a property of the corpus and not of the order its documents arrived in.



the merge, all the way down — commutative and associative at every level

`Tables.merge = mergeBy Path.lt over TableObs.merge`

`TableObs.merge = counts add, flags disjoint, members by mergeBy over Obs.merge`

`Obs.merge = counts add, flags disjoint, seen by unionBy Ty.lt`

Tables, members and the set of observed types are all kept in order (`Tatami/Sorted.lean`), so merging is commutative as a function and the result is canonical. Verified empirically: two independent shuffles of a 1,000-document corpus give a byte-identical report.

- **Obs** holds the set of types seen, not their running join. The join is taken once at the end, by `Obs.joined`. A type conflict is therefore found when the set is complete, and can name every type involved rather than the two that happened to meet first.
- **Nothing downstream sorts.** That matters because `Array.qsort` has no correctness lemmas in core — not even that it returns a permutation of its input.
- **Reading is bounded.** `Stream.foldDocuments` takes 1 MiB chunks and decodes only whole documents, so memory is a chunk plus one document plus the accumulator. Standard input is the exception: a pipe has no size to bound a loop with, so it is read whole.
- **orderTables** is the load order. It topologically sorts the schema along the graph the foreign keys run down, so a table always follows the tables it points into — by construction rather than by care.

03

What gets generated

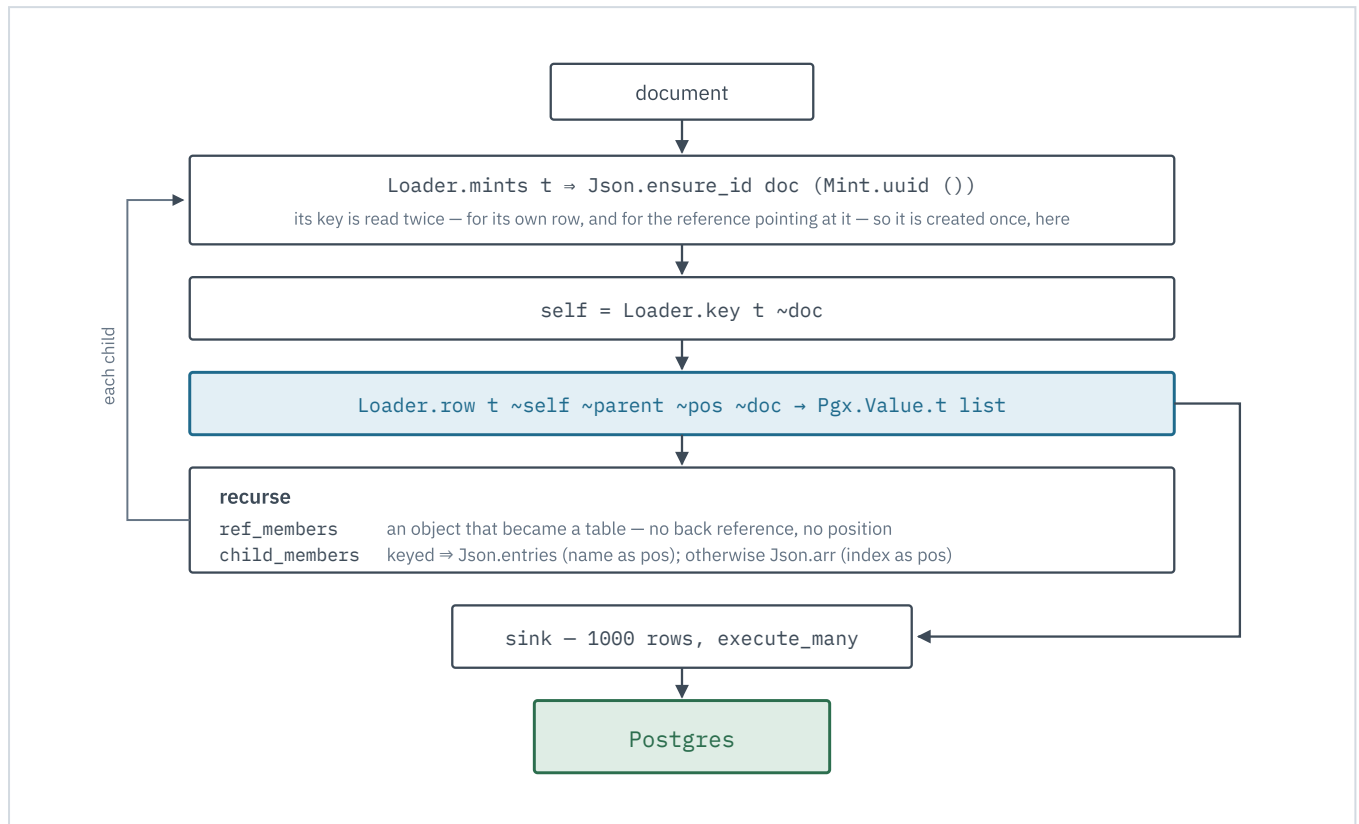
Three modules per table, and one registry. `Loader` is the only thing `bin/load.ml` talks to — which is why that file contains no table name, and adding a table to the corpus does not change it.

UNIT	FROM	HOLDS
<code>repo.mli</code>	<code>genModule</code>	<code>type id</code> (a transparent alias), <code>make</code> (labelled), <code>get</code> , <code>of_<parent></code> , getters, setters
<code>repo.ml</code>	<code>genModule</code>	The record type and those functions
<code>repo_row.ml</code>	<code>genLoader</code>	<code>columns</code> , <code>key ~doc</code> , <code>row ~self ~parent ~pos ~doc</code>
<code>loader.ml</code>	<code>genRegistry</code>	<code>tables</code> , <code>root</code> , <code>ddl</code> , <code>constraints</code> , and per table: <code>columns</code> , <code>keyed</code> , <code>mints</code> , <code>child_members</code> , <code>child_tables</code> , <code>ref_members</code> , <code>ref_tables</code> , <code>key</code> , <code>row</code>

The loaders deliberately have no `.mli`, so `Tatami.Schema.load_dir`, which reads `schema/*.mli` as data, still sees four tables rather than nine. Every `match` in the registry has a default, so asking about a table that does not exist is answered rather than a crash.

Pass 2 — the walk

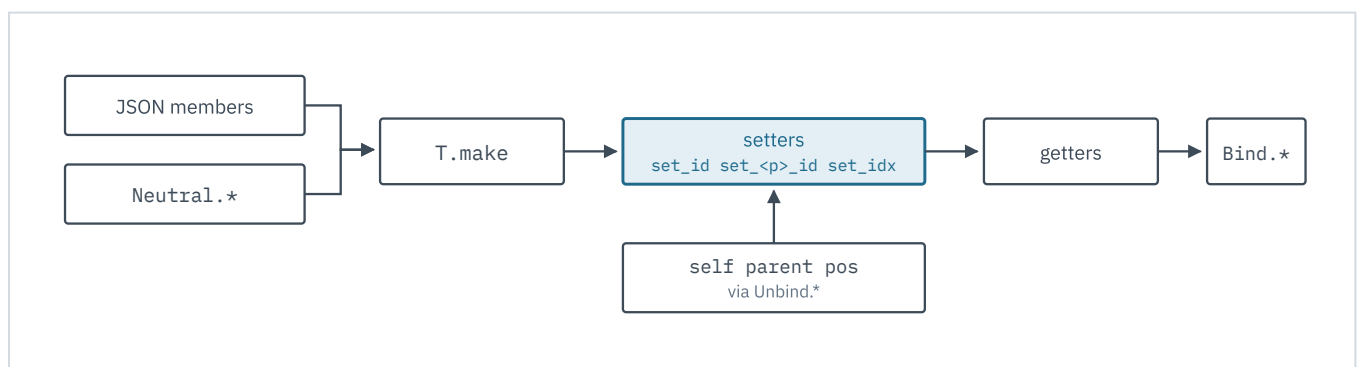
Three things the JSON does not carry are supplied here: the key of a document that has none, the key of the parent, and the position within a collection. They are exactly the three the layout marks as derived.



Everything table-specific is asked of `Loader`. What is left in `bin/load.ml` is the part that is the same for every schema: descend the document, thread each row's key down to the rows that point at it, hand batches to the database.

Where the generated modules do their work

Rows are not bound straight from JSON. Each row is built as a typed record through `make`, completed with setters, and read back out through the getters. The record is what reaches the database — `pgx` never sees the JSON.



From `schema/run_row.ml`, generated by `genLoader`.

```
(* schema/run_row.ml *)
let row ~self ~parent ~pos ~doc =
  let r = Run.make ~id:Neutral.int ~repo_id:Neutral.uuid ~idx:Neutral.int
    ~branch:(Json.string doc "branch") ~ms:(Json.int doc "ms")
    ~status:(Json.string doc "status")
    ~trigger:(Json.string_opt doc "trigger") in
  let r = Run.set_id      r (Unbind.int self) in
  let r = Run.set_repo_id r (Unbind.uuid parent) in
  let r = Run.set_idx     r (Unbind.int pos) in
  [Bind.int (Run.id r); Bind.uuid (Run.repo_id r); Bind.int (Run.idx r);
   Bind.string (Run.branch r); Bind.int (Run.ms r);
   Bind.string (Run.status r); Bind.string_opt (Run.trigger r)]
```

What the setters are actually for

Ten of the twenty-five generated setters are **ever called**, and they are the same three per table: `set_id`, `set_<parent>_id`, `set_idx`. Every member setter — `set_name`, `set_branch`, `set_status` — is dead, because member values go straight into `make` from `Json.*`. Nothing ever updates a loaded row. So `make` writes the data; the setters patch in the three values the walk knows and the document does not.

The record is a typecheck, not a carrier

It lives for one expression and is immediately destructured. Its value is that `make` is labelled, so a column that is missing, misnamed or of the wrong type fails at `ocamlc` rather than at `insert`. That is real, and it is most of what the `.mli` earns on this path — but it is a compile-time guarantee, not a runtime one.

LIB/SHRED HAND-WRITTEN, SCHEMA-INDEPENDENT

Json	Reads a member by its <i>unmangled</i> JSON name; an absent member and an explicit null are the same thing. The only place the two spellings meet: <code>Json.bool doc "private"</code> filling a column called <code>private_</code>
Bind	An OCaml value as a bound <code>Pgx.Value.t</code>
Unbind	A bound value back as an OCaml value, at the type the layout named
Neutral	What a derived column holds between <code>make</code> and the setter that stamps it. None of these reaches the database
Mint	An id for a document that carries none. Called by the walk, not by the generated loaders

06

Running it

There is no schema file to apply before loading. The tables, their columns, their nullability and their foreign keys are all derived from the corpus, and `load.exe` issues them over the same connection it loads on.

```
lake build
./lake/build/bin/tatami CORPUS --config CONFIG -o schema ; generate
./db/up.sh ; postgres container
./db/load.sh CORPUS ; dune build, then load
```

The config is not checked in. Without one, table names are path-derived — `root` , `runs` , `runs_jobs` , `runs_jobs_steps` . The committed `schema/` uses `repo` , `run` , `job` , `step` , so it was generated with a config that no file in the repository records. This one reproduces it exactly:

```
{ "root": "repo",
  "names": { ".runs[]": "run",
             ".runs[].jobs[]": "job",
             ".runs[].jobs[].steps[]": "step" } }
```

Verified: `tatami corpus/small.json --config <above> -o <dir>` produces files byte-identical to every one of the thirteen committed under `schema/` . Worth adding to the repository, since the committed OCaml cannot otherwise be regenerated.

GENERATE	PEAK RSS	UNITS OUT	ROWS IMPLIED
0.33 s	71 MB	13	26,329
CORPUS/SMALL.JSON	GENERATOR	FOR 4 TABLES	FROM 1,000 DOCUMENTS

Measured on `corpus/small.json` : 1,000 repos, 2,511 runs, 5,074 jobs, 17,744 steps. End-to-end load timings against Postgres are *not* reported here — the figures in the earlier note (20.3 M rows, 360 s shred, 109 s load) were measured on the discarded `COPY` path and do not describe this one.

07

What is not wired

THING	STATE
get	<code>schema/repo.ml</code> has <code>let get _ = failwith "Repo.get: no row source"</code> . Writing goes through the generated modules; reading back does not go through them at all. If the round trip is to be typed, this is the missing piece HOLE
member setters	Generated, never called. They cost nothing, but they are not evidence of a mutation path, because there is not one DEAD
pgx read types	<code>pgx</code> returns <code>character</code> where an <code>integer</code> is expected (see <code>notes.org</code>). On the read side, so it does not affect loading OPEN
the config	Not in the repository, though the committed <code>schema/</code> depends on it. Recovered in <code>\$06</code> MISSING
<code>Path.parseGo</code>	The last <code>partial def</code> in the Lean codebase OPEN

THING	STATE
<code>infer_perm</code>	Restructured so the property is structural; the remaining chain is <code>mergeBy</code> commutative and associative, discharged for <code>String</code> , <code>Path</code> , <code>Ty</code>

08

What changed since the earlier note

	EARLIER NOTE	NOW
SQL	Lean wrote <code>schema.sql</code> and <code>constraints.sql</code>	Lean writes OCaml holding SQL strings
Loading	Shell script over <code>psql</code> and <code>COPY</code>	<code>load.exe</code> over <code>pgx</code> , batched <code>insert</code>
Rows	Written as TSV, bound by Postgres	Built through <code>make</code> + setters, read out by getters
Passes	Two passes by one program	Two passes by two programs
Table names	Path-derived only	<code>names</code> config, path-derived as fallback
Registry	None — the loader knew the tables	Generated <code>Loader</code> ; <code>bin/load.ml</code> names no table